

TypeScript Important Concepts:

1. Interface vs Type (TypeScript)

Interface

Object shape define karne ke liye use hota hai.

```
interface User {  
  name: string;  
  age: number;  
}
```

Type

Kisi bhi type ko define kar sakta hai.

```
type User = {  
  name: string;  
  age: number;  
};
```

Difference

Interface extend karna

```
interface Animal {  
  name: string;  
}  
  
interface Dog extends Animal {  
  breed: string;  
}
```

Type extend karna

```
type Animal = {  
  name: string;  
};  
  
type Dog = Animal & {
```

```
    breed: string;  
};
```

Interface merging support karta hai

```
interface User {  
    name: string;  
}
```

```
interface User {  
    age: number;  
}
```

Becomes:

```
{  
    name: string;  
    age: number;  
}
```

Type merging nahi karta

```
type User = { name: string };  
type User = { age: number };
```

✗ Error.

Interview Answer:

Interface mainly objects aur contracts define karne ke liye use hota hai, jabki type aliases objects ke saath unions, tuples aur primitive aliases bhi define kar sakte hain. Large React projects me main interfaces ko API models ke liye aur types ko unions aur utility compositions ke liye prefer karta hoon.

2. What are Generics?

Generics = reusable code with type safety.

Without generics:

```
function identity(value: any) {  
  return value;  
}
```

Type safety lost.

With generics:

```
function identity<T>(value: T): T {  
  return value;  
}
```

Usage:

```
identity<string>("hello");  
identity<number>(42);
```

React example:

```
const [user, setUser] = useState<User | null>(null);
```

User generic type parameter ki tarah kaam kar raha hai.

Interview Answer:

Generics allow us to write reusable components and functions while preserving type information. They improve flexibility without sacrificing type safety.

3. Why TypeScript over JavaScript?

JavaScript:

```
function add(a, b) {  
  return a + b;  
}
```

```
add(10, "20");
```

No error.

TypeScript:

```
function add(a: number, b: number) {  
  return a + b;  
}
```

Compiler catches errors.

Benefits:

1. **Early error detection** : Compile time.
2. **Better autocomplete**: IDE support.
3. **Easier refactoring**: Large applications me helpful.
4. **Self-documenting code**: Types explain intent.

Interview Answer:

TypeScript provides static typing, better tooling, safer refactoring, and catches many errors during development rather than runtime, making it particularly valuable in large-scale React applications.

4. Utility Types

Built-in TypeScript helpers.

Partial

Sab optional.

```
interface User {  
  name: string;  
  age: number;  
}
```

```
type UpdateUser = Partial<User>;
```

Result:

```
{  
  name?: string;  
  age?: number;  
}
```

Pick: Kuch properties choose karo.

```
type UserPreview = Pick<User, "name">;
```

Omit: Kuch properties hata do.

```
type UserWithoutAge = Omit<User, "age">;
```

Record: Key-value mapping.

```
type UserRoles = Record<string, string>;
```

Interview Answer:

Utility types help transform existing types efficiently. Common examples include Partial, Pick, Omit, and Record, which reduce duplication and improve maintainability.
